Due Diligence Research Agent

Technical Writeup — Together Fund Internship Assignment

Kushagra | kushagrabhargava93@gmail.com | July 2026

1. Problem

A Together Fund partner receiving a pitch needs to answer three questions before the next call: Is the market real? Is the founder credible? Is the traction believable? Today that means 3–4 hours of manual Googling, LinkedIn checks, and Crunchbase dives — multiplied across 8+ deals a week. The Due Diligence Research Agent eliminates that manual loop.

2. What It Does

The partner types a company name, one-line description, and the founder's LinkedIn URL. The agent runs a 5-step autonomous research loop — searching competitors, funding rounds, founder background, market size, and recent news. After each step, the LLM reads what was just found and decides what to investigate next. The loop ends with a structured 2-page memo covering Market, Competition, Founder, Traction, Red Flags, and a Verdict with confidence score. Every reasoning step is visible live in the UI as the agent works.

3. Architecture

Stack: Python 3.11, Streamlit, Groq API (llama-3.3-70b-versatile), Tavily Search API, Pydantic v2.

File structure:

`app.py (Streamlit) → agent/core.py (agentic loop) → agent/tools.py (Tavily search + disk cache) → agent/prompts.py (LLM prompt templates) → agent/memo_schema.py (Pydantic DDMemo model)`

The agentic loop in core.py runs for 5 steps. At each step, it passes findings_so_far into a next-query prompt — the LLM reads all prior search results and outputs one targeted search query. This is what makes it genuinely agentic: each step conditions on observed results from previous steps, not a fixed script. The final synthesis call reads all 5 findings and outputs a structured JSON memo validated by Pydantic.

4. Why No LangChain / LangGraph

An agentic loop needs exactly three primitives: a loop, a decision (what to do next), and a tool call. All three are implemented directly in ~60 lines of core.py. LangGraph wraps these same primitives in a graph abstraction that adds overhead, obscures the reasoning surface, and signals that the engineer cannot build agents from scratch. For a technical evaluation, the scratch-built version is the correct choice.

5. Key Technical Decisions

Tavily over DuckDuckGo: DDG rate-limits aggressively at the IP level during repeated demo runs. Tavily is purpose-built for LLM agents — structured results, no blocking, 1000 free searches/month.

Groq over Anthropic: Groq's free tier with llama-3.3-70b-versatile is fast enough for a live demo (sub-2s per step) and has no cost barrier. The agent architecture is model-agnostic — swapping back to Claude requires changing one line.

Disk cache: Every successful search result is cached to cache/ as a JSON file keyed by MD5 hash of the query. Demo re-runs are instant and never touch the network. Failed results are auto-invalidated and retried.

Streamlit over React/FastAPI: The agentic loop is the interesting part, not the UI. Streamlit's st.empty() gives a live-updating log panel with zero frontend overhead. The two-column layout (reasoning log left, memo right) makes the agent's thinking immediately legible to a non-technical partner.

Session persistence: Each completed DD run is saved to sessions/ as JSON with the full findings log and memo. Easy to demo run history.

6. Limitations & Production Path

Early-stage startups with thin public footprint return Low confidence memos — the agent flags this explicitly rather than hallucinating. In production, the search layer would be augmented with Crunchbase API and LinkedIn data partners for richer signal.

The fixed 5-step loop works well for demo but a production version would use a dynamic loop with a stopping condition (e.g. confidence threshold reached or max steps hit). The Pydantic memo schema makes this straightforward to add.

At Together Fund's scale (8+ deals/week), this saves roughly 25–30 partner hours weekly and ensures no deal slips through without a baseline research pass.